Experiment No. 6 – Advanced Jenkins Pipeline and Job Management

Practical Assignment: Exploring Jenkins Jobs, Parameters, Environment Variables, Notifications and Pipeline Control

Course: Introduction to DevOps / Cloud Computing

Topic: Advanced Jenkins Pipeline

Level: Intermediate

Course Teacher: Mr. Ajit B. Patil

---

1. Aim

To explore advanced Jenkins features by creating a parameterized pipeline and using environment variables, conditional stages, build parameters, and pipeline post-actions.

---

2. Objectives

By completing this experiment, students will be able to:

1. Create and configure different types of Jenkins jobs.
2. Understand Jenkins Pipeline and Jenkinsfile.
3. Use **environment variables** in Jenkins.
4. Create **parameterized builds**.
5. Implement **conditional stages**.
6. Use Jenkins **post-actions**.
7. Understand build status and pipeline control.
8. Explore Jenkins build history and console output.

---

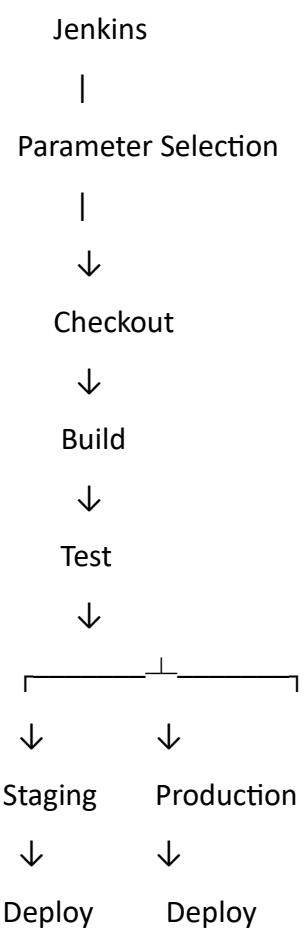
3. Introduction

Jenkins provides several features beyond basic CI/CD.

A Jenkins pipeline can be configured to:

- Accept user input.
- Use environment variables.
- Execute different stages based on conditions.
- Archive build artifacts.
- Perform actions after successful or failed builds.
- Control the application deployment environment.

Jenkins Pipeline



---

4. Prerequisites

Students should have:

- Jenkins installed and running.
- Git installed.
- GitHub account.
- Basic knowledge of Jenkins.
- Basic knowledge of Git.
- Experiment No. 5 completed.

Task 1 – Create a Jenkins Pipeline Job

Open:

http://localhost:8080

Click:

New Item → Pipeline

Enter:

Advanced-Jenkins-Pipeline

Select:

Pipeline

Click **OK**.

Task 2 – Create a Parameterized Pipeline

Enable:

This project is parameterized

Create a **Choice Parameter**.

Parameter Name

DEPLOY_ENV

Choices

development

staging

production

This allows the user to select the deployment environment before starting the build.

Task 3 – Create Jenkinsfile

Create a file named:

Jenkinsfile

Use the following pipeline:

```
pipeline {
    agent any

    parameters {
        choice(
            name: 'DEPLOY_ENV',
            choices: ['development', 'staging', 'production'],
            description: 'Select deployment environment'
        )
    }

    environment {
        APP_NAME = 'StudentApp'
        VERSION = '1.0'
    }

    stages {

        stage('Information') {
            steps {
                echo "Application: ${APP_NAME}"
                echo "Version: ${VERSION}"
                echo "Environment: ${params.DEPLOY_ENV}"
                echo "Build Number: ${BUILD_NUMBER}"
            }
        }

        stage('Build') {
```

```
    steps {
        echo 'Building application...'
    }
}

stage('Test') {
    steps {
        echo 'Running tests...'
    }
}

stage('Deploy') {
    when {
        expression {
            params.DEPLOY_ENV == 'staging'
        }
    }

    steps {
        echo 'Deploying application to STAGING...'
    }
}

stage('Production Approval') {
    when {
        expression {
            params.DEPLOY_ENV == 'production'
        }
    }

    steps {
        input message: 'Approve production deployment?',
```

```
                ok: 'Deploy'
            }
        }

        stage('Production Deploy') {
            when {
                expression {
                    params.DEPLOY_ENV == 'production'
                }
            }

            steps {
                echo 'Deploying application to PRODUCTION...'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully.'
        }

        failure {
            echo 'Pipeline failed. Check the console output.'
        }

        always {
            echo 'Pipeline execution completed.'
        }
    }
}
```

Task 4 – Understand Environment Variables

Jenkins provides predefined environment variables.

Examples:

BUILD_NUMBER

BUILD_ID

BUILD_URL

JOB_NAME

WORKSPACE

Display them using:

echo "Job Name: ${JOB_NAME}"

echo "Build Number: ${BUILD_NUMBER}"

echo "Workspace: ${WORKSPACE}"

Important Variables

Variable	Purpose
BUILD_NUMBER	Current build number
JOB_NAME	Jenkins job name
BUILD_URL	URL of the build
WORKSPACE	Workspace directory
JENKINS_URL	Jenkins server URL

Task 5 – Execute Parameterized Build

Click: **Build with Parameters**

You will see:

DEPLOY_ENV

○ development

○ staging

○ production

Select:

staging

Click: **Build**

Task 6 – Observe Pipeline Execution

The pipeline executes:

Information

↓

Build

↓

Test

↓

Deploy

For staging, the production stages are skipped.

The Stage View should look similar to:

Information SUCCESS

Build SUCCESS

Test SUCCESS

Deploy SUCCESS

Production SKIPPED

Task 7 – Test Production Deployment

Run the pipeline again.

Select: production

The pipeline executes:

Information

↓

Build

↓

Test

↓

Production Approval

↓

Production Deploy

Jenkins pauses at:

Approve production deployment?

Click:

Deploy

The pipeline continues.

Task 8 – Test Failure Handling

Modify the Test stage:

```
stage('Test') {
   steps {
      echo 'Running tests...'
      error 'Test failed!'
   }
}
```

Run the pipeline.

Observe:

Build SUCCESS

Test FAILURE

Deploy SKIPPED

The post section executes:

Pipeline failed. Check the console output.

Restore the original Test stage after the demonstration.

Task 9 – Explore Build History

From the Jenkins dashboard, open:

Advanced-Jenkins-Pipeline → Build History

You should see:

#1 SUCCESS

#2 SUCCESS

#3 FAILURE

Click any build number to explore:

- Console Output

- Pipeline Steps
- Changes
- Build Parameters
- Workspace

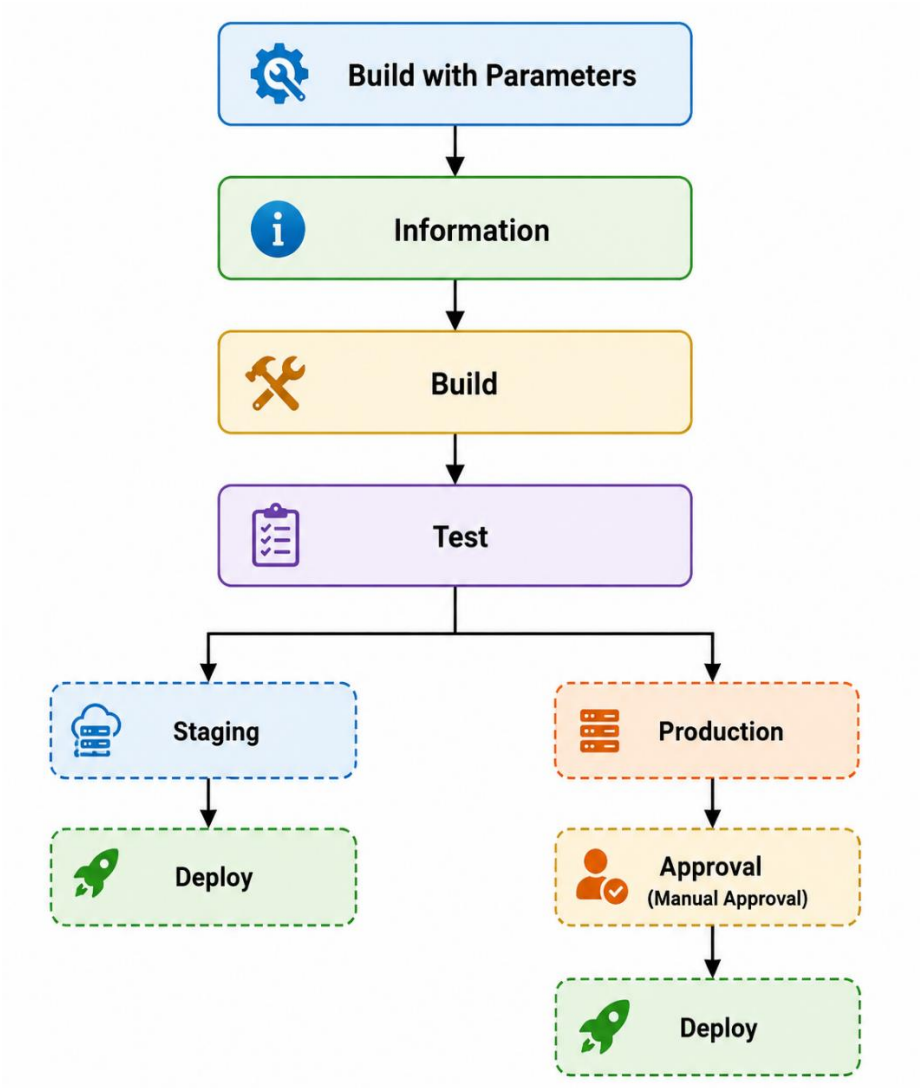
Task 10 – Explore Jenkins Workspace

Jenkins creates a workspace for every job.

The workspace contains files required during pipeline execution.

You can display the workspace using:

echo "Workspace: ${WORKSPACE}"



Concepts Learned

After completing this experiment, students should understand:

1. Jenkins Pipeline

Pipeline as Code using a Jenkinsfile.

2. Parameters

Allows users to provide values before starting a build.

3. Environment Variables

Provides information about the Jenkins job and build environment.

4. Conditional Execution

The when block controls whether a stage executes.

5. Manual Approval

The input step can pause a pipeline until a user approves it.

6. Post Actions

The post block performs actions after pipeline execution.

success

failure

always

7. Build History

Jenkins maintains the history of every pipeline execution.